

Hacking CSRF Tokens using CSS History Hack

Hacking CSRF Tokens using CSS History Hack - Author not stated, securethoughts.com.

- Published: date not stated
- Original: <<http://securethoughts.com/2009/07/hacking-csrf-tokens-using-css-history-hack/>>
- Current location: <<https://securethoughts.com/2009/07/hacking-csrf-tokens-using-css-history-hack/>>
- Preserved from: <https://securethoughts.com/2009/07/hacking-csrf-tokens-using-css-history-hack/> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Hacking CSRF Tokens using CSS History Hack | Secure Thoughts

Hacking CSRF Tokens using CSS History Hack

Update: Security researchers [Sirdarckcat](#) and [Gareth](#) were kind enough to share the code for a pure CSS based CSRF token finder [here](#) . This is stealthier than my PoC below, which used a combination of both JS and CSS. So, it will still work even if you disable javascript and you are not safe anymore [\[image: :\(\)\]](#) . To make this PoC more responsive to the client, you need to use multiple CSS stylesheets using the import command. The only problem I see with this pure CSS based approach is there will be network latency involved with large key spaces because your large CSS stylesheet will need to be downloaded by your browser.

I was thinking about the problem of [Cross Site Request Forgery](#) and current mitigation strategies used in the Industry. In many of the real world applications I have tested so far, I see the use of random tokens appended as part of url. If the request fails to provide any token or provide a token with incorrect value, then the request is rejected. This prevents CSRF or any cross domain unauthorized function execution.

Uptil now, it was considered infeasible for an attacker to discover your CSRF token using [Brute Force Attacks](#) on the server.

The reasons being:

- It generates **lot of noise on the network and is slow**. So most probably an IDS or Web App Firewall will pick up the malicious behavior and block your ip. For example, a Base16 CSRF

token of length 5 characters (starting with a character) will generate approximately 393,216 requests.

- Many applications are programmed to **invalidate your session** after it detects more than a certain number of requests with invalid token values. E.g. 30.

I am going to change this belief by showing you a technique to quickly find csrf tokens without generating alerts. This technique is a **client side attack**, so there is almost no network traffic generated and hence, your server and IDS/Web App Firewalls won't notice it at all. This attack is based on the popular [CSS History Hack](#) found by [Jeremiah Grossman](#) 3 years ago.

In this exploit, we discover the csrf token by brute forcing the various set of urls in browser history. We will try to embed different csrf token values as part of url and check if the user has visited that url. If yes, there is a good chance that the user is either using the same CSRF token in the current active session or might have used that token in a previous session. Once we have a list of all such tokens, we can just try our csrf attack on the server using that small list. Currently this attack is feasible for tokens with length of 5 characters or shorter. I tried it on a base16 string of length 5 and was able to brute force the entire key space in less than 2 minutes.

Some of the prerequisites for this attack to work are either

- CSRF token remains the same for a particular user session. e.g. csrf token=hash(session_id) OR
- CSRF token submitted in older forms for the same session is accepted. Many times, this is the case as it enhances user experience and allows using forward and back browser buttons.

Proof of Concept is available [here](#). Before running the PoC, you need to change the url and csrftoken parameter values.

For testing using the defaults, you need to first visit one of the following urls, e.g.

- <http://securethoughts.com/?param1=val1&csrftoken=b59fe> [change b59fe to any 5-digit base 16 string starting with a character, i.e. greater than a0000]
- <http://tinyurl.com/l2lwgd> [which is 301 redirect to previous url].

Note: <http://www.securethoughts.com> and <http://securethoughts.com> are treated differently while storing in browser history.

For making this attack unfeasible,

- **Server-Side Solution (for developers):**

- Make your CSRF tokens long enough (8 or more chars) to be unfeasible for a CLIENT SIDE attack. The ever-increasing processing power will make this attack feasible for longer tokens as well.
- Store your CSRF token as part of hidden form field, rather than putting in url.
- Use a different random token for every form submission and not accept any obsolete token, even for the same session.

- **Client-Side Solution (for your customers/users):**

- Use a browser plugin such as [SafeHistory](#), which defends against visited-link-based tracking techniques.
- Use the private browsing mode in your browser.

And last, but not the least, XSS obliterates all the CSRF protections possible. So, get rid of XSS first.
I would like to thank [Jeremiah](#) for providing his insightful feedback on this post.

**** Category:** [Latest Articles](#)

**** Tagged:**

Archived from <http://securethoughts.com/2009/07/hacking-csrf-tokens-using-css-history-hack/>. Rendered offline from the local Markdown copy.